Documentación Proyecto ERASMUS

Sistema de Medición de Índice UV

Contenido

Descripción del Proyecto 3
Sensor ML8511 3
Acondicionamiento 3
Manejo de la Señal 4
Comunicación Serial. 4
Notas Importantes del Código 4
4 Mensajes por Comando y Cómo Obtenerlos 5
Captura de Datos con Wireshark 6
Notas sobre el Soporte y Visión a Futuro 7

Descripción del Proyecto

El proyecto consistió en realizar un sistema de medición de índice UV para lograr establecer un sistema similar al solmáforo disponible en la Universidad con una pantalla de LEDs comunicada a través de WI-FI. El sensor utilizado es el ML8511, el reto estaba en acondicionar la señal y lograr que la pantalla despliegue de manera condicional los mensajes según las mediciones.

Sensor ML8511

Acondicionamiento

Lo clave en el sensor ML8511, es que la escala que se establece en la hoja de datos de este no tiene un sentido que pueda adaptarse al objetivo del proyecto, por ello solo se debía sincronizar los datos que se obtenían con los datos del solmáforo, en este caso, la función resultante se basa en la curva ofrecida, pero tiene ajustes determinados por mediciones in situ comparándolas con la del solmáforo, esta función está embebida en la ESP32 que realiza las mediciones.

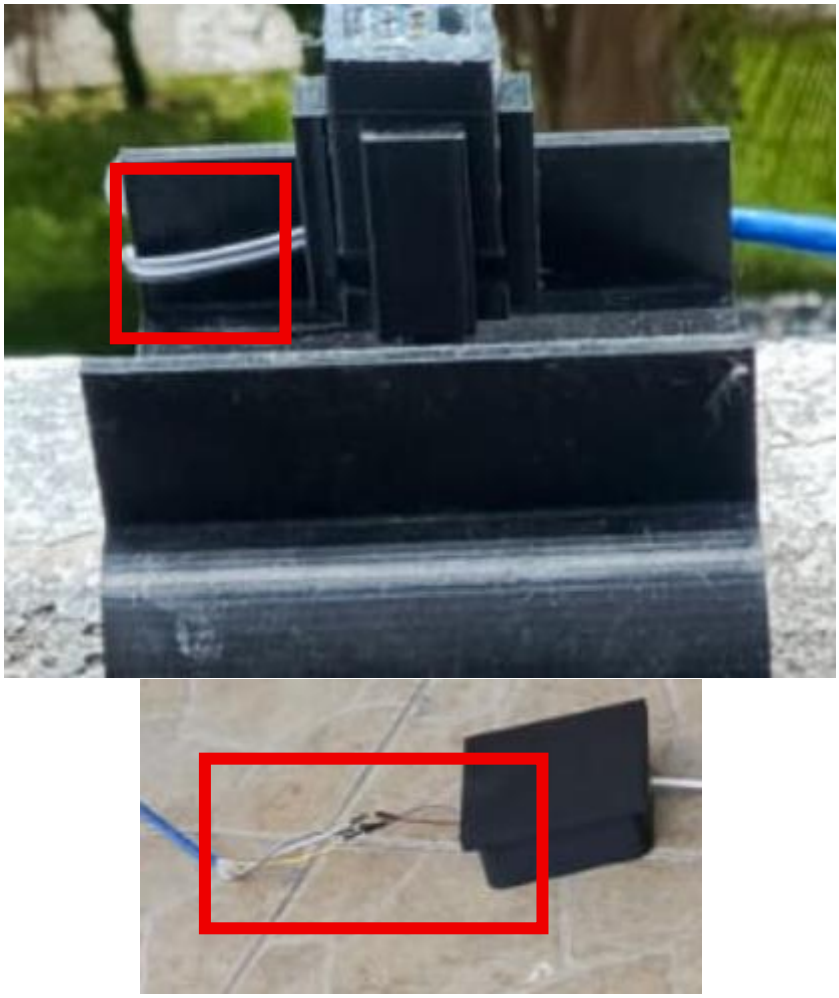


Fig. 1.: Conexiones del Sensor

De lo observado en la Figura 1, los tres cables necesarios para el sensor se encuentran de cierta forma, en código de colores: en la primera imagen que es la salida del sensor, se tiene un cable blanco (VCC), un cable plomo (señal analógica de Voltaje) y un cable negro (GND); para la llegada a la caja de la ESP32 los cables son: Amarillo (VCC), blanco (señal analógica de Voltaje) y azul (GND). El cable por el que están unidas es un cable UTP de Ethernet.

Manejo de la Señal

El código fuente está en el repositorio de github:
https://github.com/CarlosNav483/Subsistema-4.git

En la ESP32 se hace un filtro mediante promedio para obtener una medición cada minuto, de esta manera la señal no se actualiza demasiado rápido como para que no le dé tiempo a refrescar la imagen de la pantalla.

```
int averageAnalogRead(int pinToRead)
{
  byte numberOfReadings = 8;
  unsigned int runningValue = 0;

  for(int x = 0 ; x < numberOfReadings ; x++)
    runningValue += analogRead(pinToRead);
  runningValue /= numberOfReadings;

  return(runningValue);
}
```

Fig. 2.: Función para Promediar la Lectura Analógica

```
float uvIntensity = (outputVoltage - 0.78)*(6.8182*2)*1.6; //mapflc
//float indexflt = mapfloat(outputVoltage, 0.0, 15.0, 0.0, 12.0);
int uvIndex = int(uvIntensity);
```

Fig. 3.: Ecuación del Sensor

Con esta ecuación se obtuvieron los resultados más cercanos a la medición con el solmáforo.

Comunicación Serial.

Para la comunicación serial en este caso se realizó 115200 baudios y únicamente con la función de debug del programa o mediciones en tiempo real.

Notas Importantes del Código

El código implementado podría replicarse para añadir otra pantalla, sin embargo, cabe destacar que la conexión WI-FI requiere de un SSID y la contraseña como ejemplo está la pantalla implementada:

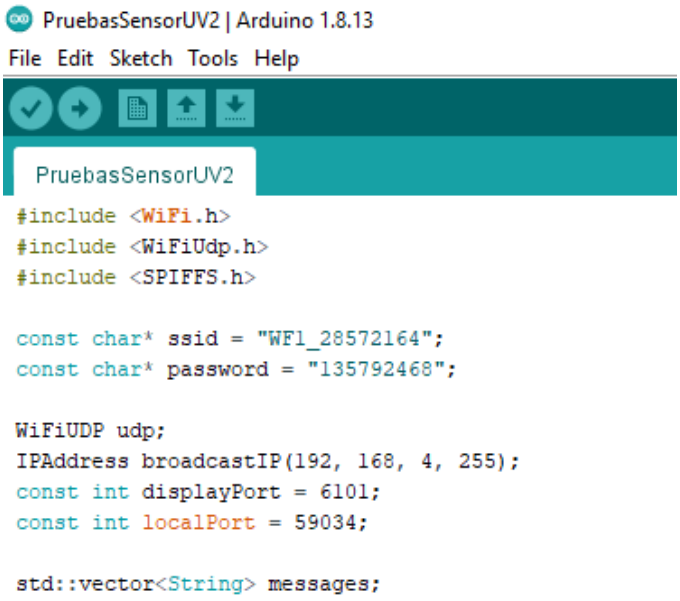


Fig. 4.: Inicio del Código

Una particularidad de esta implementación es que debe manejarse con una versión 1.8.13 de Arduino IDE o inferior, pues en esta (actualmente es la única forma) se puede acceder a la memoria de SPIFFS de la ESP32. Como los datos eran demasiado largos como para cargarlos en la memoria de código de la ESP32, una alternativa muy conveniente fue usar estos SPIFFS que son espacios de memoria para almacenar páginas web, que en este caso almacenarían los 48 mensajes necesarios para ordenar a la Pantalla LED que cambie de mensaje.

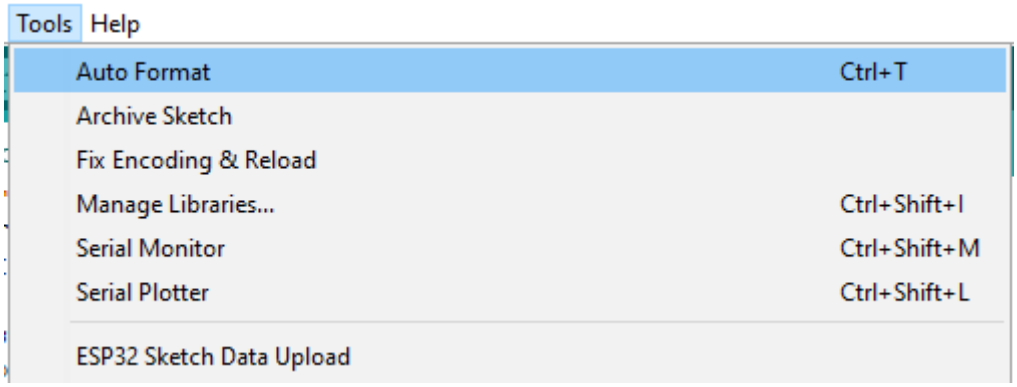


Fig. 5.: Pestaña Tools

La opción para esto es ESP32 Sketch Data Upload, esta opción no está disponible en versiones superiores por el momento y resulta ser una herramienta bastante útil en esta aplicación.

4 Mensajes por Comando y Cómo Obtenerlos

Los mensajes ya se encontraban predefinidos por el fabricante en su aplicación especializada para el manejo de la pantalla:

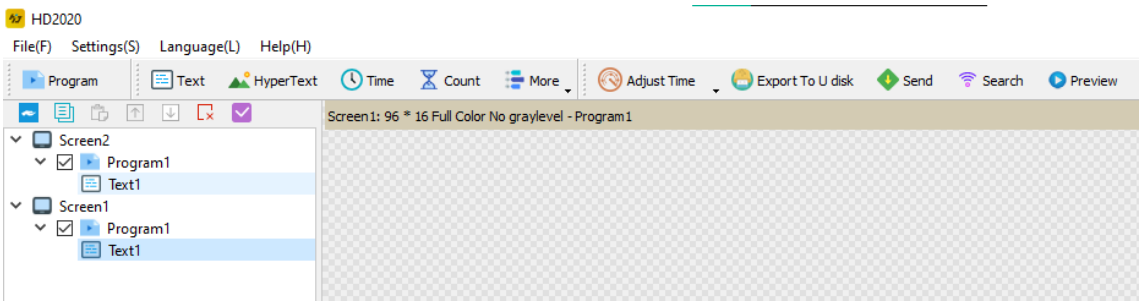


Fig. 6.: Pantalla Principal de la Aplicación

Como el fabricante es LEDArt y sus discos están disponibles en el laboratorio, la única alternativa de conexión fue interceptar los mensajes que esta proveía desde la computadora para automatizarlos y mandarlos condicionalmente con la ESP32. El protocolo por el que se comunica es UDP, esto no influye demasiado si solo se tratará de replicar los comandos. Se debe realizar mediante wireshark y como es una conexión única de WI-FI (como conectarse a un router), la intercepción de datos resulta ser relativamente sencilla.

Captura de Datos con Wireshark

Una vez descargada la aplicación de Wireshark (actualmente gratuita), en la pantalla principal se deberá colocar en capturar Wi -Fi:

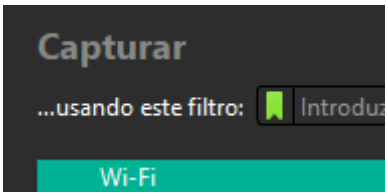
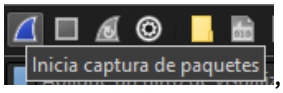


Fig. 7.: Elección de Captura

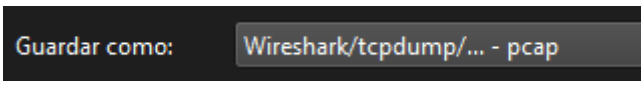
Luego, se debe enviar el mensaje requerido mediante la aplicación HD2020 y luego al mismo tiempo presionar en el ícono de iniciar captura de paquetes 

, entonces se obtendrá algo similar a lo mostrado en la Figura 8:

	Time	Source	Destination	Protoc
1	0.000000	192.168.100.11	224.0.0.251	MDNS
2	0.000434	192.168.100.11	224.0.0.251	MDNS
3	1.600111	192.168.100.11	224.0.0.251	MDNS
4	1.600551	192.168.100.11	224.0.0.251	MDNS

Fig. 8.: Paquetes Capturados.

Con ello listo, se deberá presionar que es la opción de guardar captura. Y se deberá guardar como tcpdump – pcap.



Y luego se deberá pasar mediante CMD esta captura a texto y adjuntarla a la carpeta del programa de Arduino en una carpeta llamada data. Luego se hará uso de la herramienta mencionada en el apartado anterior y esto será todo.

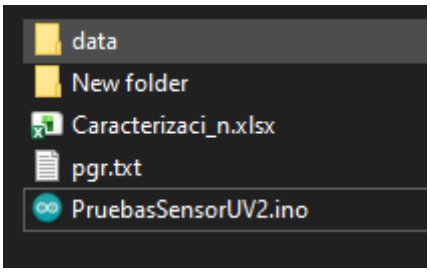


Fig. 9.: Carpeta del Programa de Arduino

En CMD corriendo como administrador y en la carpeta de destino de tshark (se encuentra en la carpeta donde se instaló Wireshark) se insertará el comando: tshark -r C:/Destinodecaptura.pcapng -T fields -e data > nombre_de_destino.txt

```
C:\Program Files\Wireshark>: tshark -r C:\Users\Calouch\Documents\Proyecto ERASMUS\capturaa1.pcap -T fields -e data > C:
\Users\Calouch\Documents\Proyecto ERASMUS\nombre_de_destino.txt
C:\Program Files\Wireshark>
```

Notas sobre el Soporte y Visión a Futuro

Es importante mencionar que el soporte situado en el balcón del laboratorio, está pensado en ser modular para facilitar encajes de piezas y mejoras tan pronto como se reporten situaciones adversas. El techo fijado al sensor se estableció como no útil pues se requiere de una inclinación y además se pensó que tal vez se podría implementar un sistema de control de posición para inclinar el sensor a seguimiento del sol y obtener así una mejor medición del índice UV a lo largo del día. Tanto el techo como el sensor están situados de forma que se pueda desarmar, el sensor es encajable y el techo es diseñado mediante corredera. Todo el material de impresión está en la memoria del laboratorio.

Fig. 10.: Entrega Final

Es importante destacar que para este equipo es necesaria la conexión a 120VAC para el controlador (mediante adaptador) y la pantalla además de una comprobación del alcance de la señal Wi -Fi para comunicación.